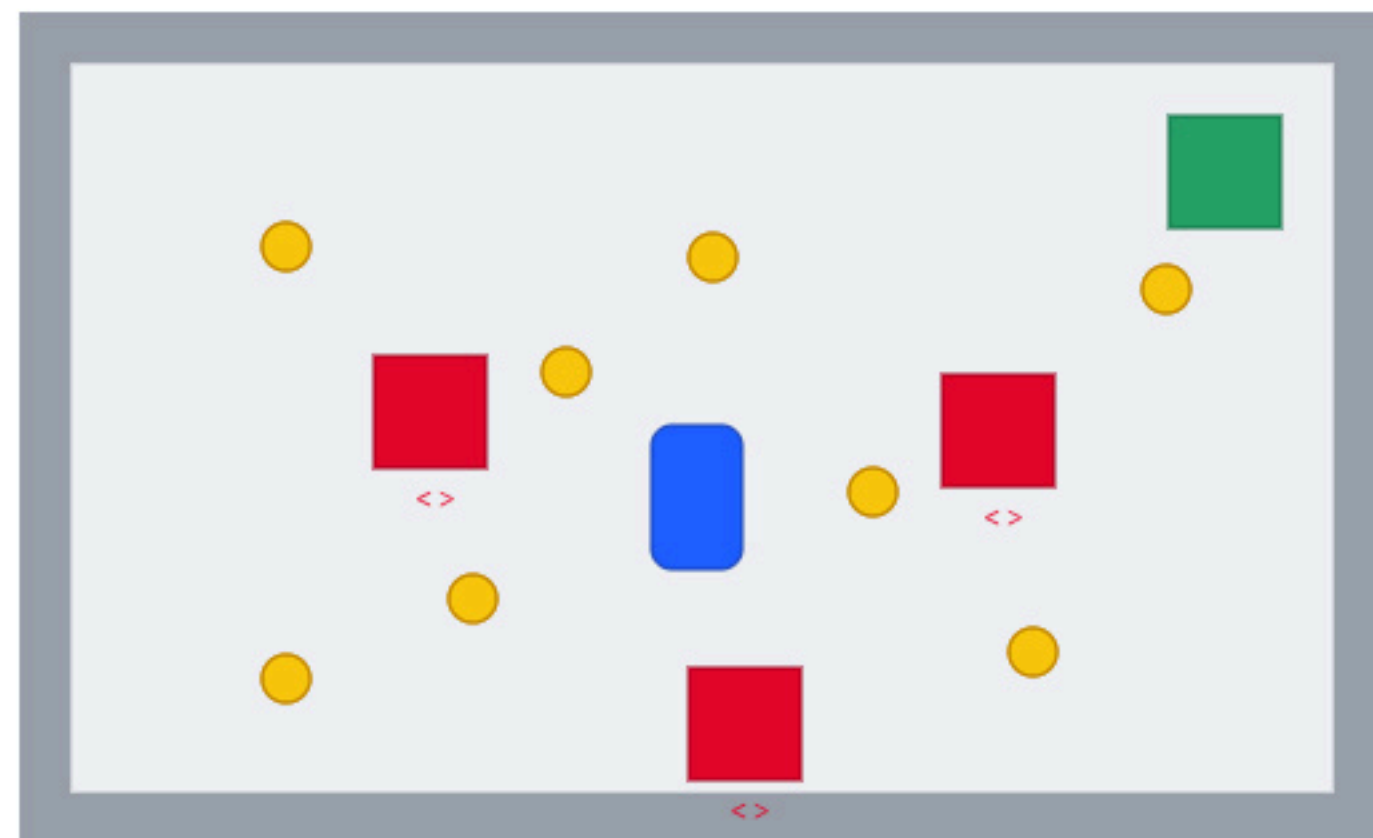


Cube Runner Arena

Clase practica de IA aplicada a Unity 3D y C#

5 horas / ChatGPT gratuito / primitivas basicas de Unity



Que construiremos hoy

- Un minijuego 3D completo con capsula, cubos, esferas y plano.
- Movimiento, coleccionables, score, tiempo, obstaculos, victoria y derrota.
- Un flujo de trabajo con ChatGPT gratuito: pedir, integrar, probar y corregir.
- Una base de prototipo reutilizable para mecanicas arcade sencillas.

La regla de la practica

ChatGPT ayuda, pero Unity verifica.

Cada respuesta generada se prueba en Play Mode. El alumno debe revisar Inspector, componentes, tags, colliders, Rigidbody y version de Input.

no copiar a ciegas

cambios pequenos

verificar en escena

Ruta de 5 horas

Apertura

20 min

Escena base

30 min

Player

40 min

Coleccionables

40 min

Tiempo/estados

40 min

Obstaculos

45 min

Camara y pulido

35 min

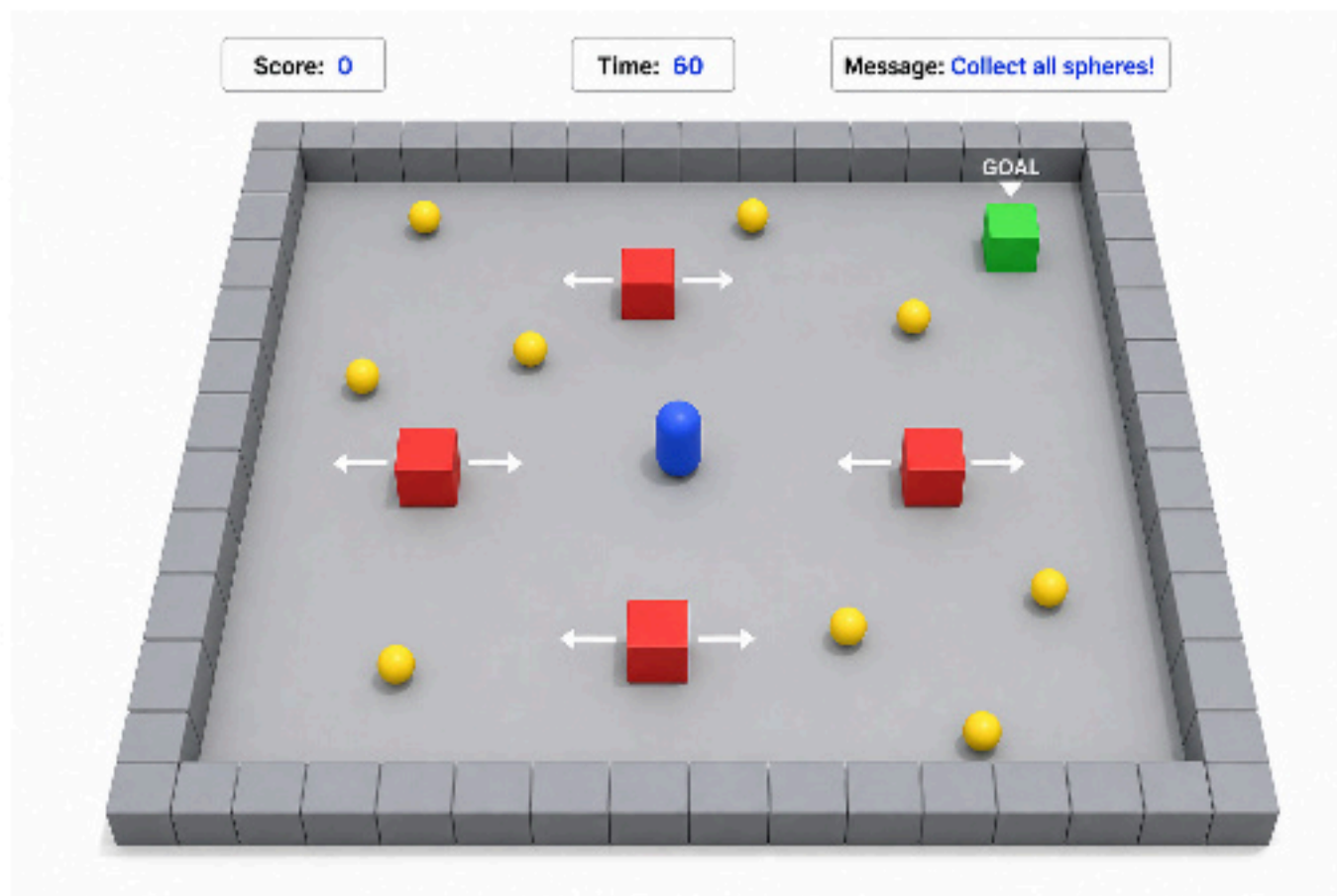
Debugging IA

30 min

Entrega

20 min

Resultado jugable



- Capsule azul: jugador.
- Spheres amarillas: coleccionables.
- Cubos rojos: obstaculos moviles.
- Cubos grises: paredes.
- Cubo verde: referencia de goal.

Prompt maestro

Actua como desarrollador Unity C# senior.
Estoy creando Cube Runner Arena con primitivas basicas:
capsula, cubos, esferas y plano.

Restricciones:

- Unity 3D, C#
- ChatGPT gratuito, sin suscripcion
- no assets externos, DOTween ni Cinemachine
- no nuevo Input System salvo que lo pida
- cambios pequenos, scripts claros
- dime que revisar en Inspector

Antes de codigo: lista GameObjects, componentes y riesgos.

Escena base en Unity

Ground

Plane escalado. Collider por defecto.

Player

Capsule + Rigidbody + tag Player.

Collectibles

Spheres con Collider en Is Trigger.

Obstacles

Cubos rojos con Collider normal.

Canvas

ScoreText, TimeText, MessageText.

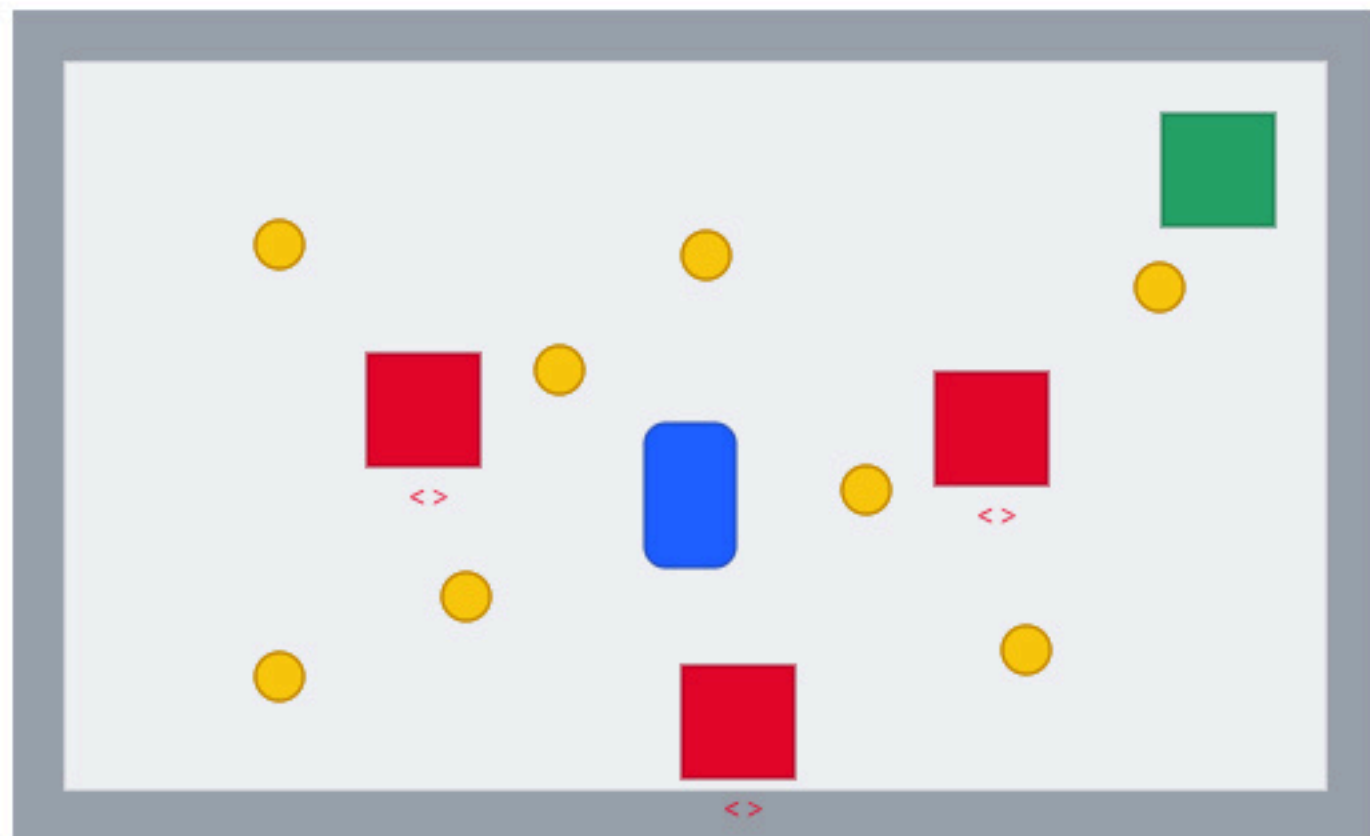
GameManager

Estado, puntuacion, tiempo y reinicio.

Main Camera

Vista inclinada o seguimiento simple.

Terreno de juego



- La escena debe ser legible antes de programar.
- El layout crea decisiones de dificultad: riesgo, distancia y tiempo.
- Las primitivas bastan para prototipar mecánicas reales.

Bloque 1: configurar escena

Prompt:

Estoy preparando una escena basica de Unity 3D para Cube Runner Arena.

Uso solo plano, capsula, esfera y cubo.

Dame una checklist corta para:

- player con Rigidbody
- suelo con Collider
- paredes
- camara
- materiales
- jerarquia

No generes codigo todavia.

- Aprendizaje: no todo prompt debe pedir codigo.
- Primero ordenamos escena y dependencias.
- Despues programamos con menos incertidumbre.

Movimiento del player

Input

WASD o flechas sobre XZ.
Se lee en Update.

Fisica

Rigidbody.MovePosition.
Se aplica en FixedUpdate.

Inspector

Move Speed editable.
Congelar rotacion X/Z.

Prompt:

Necesito PlayerMovement para Unity 3D.

Player = Capsule con Rigidbody.

Movimiento en plano XZ.

Usa Rigidbody.MovePosition en FixedUpdate.

No uses el nuevo Input System.

Velocidad con SerializeField.

Codigo completo + Inspector.

PlayerMovement.cs

```
using UnityEngine;

[RequireComponent(typeof(Rigidbody))]
public class PlayerMovement : MonoBehaviour
{
    [SerializeField] private float moveSpeed = 6f;

    private Rigidbody rb;
    private Vector3 moveDirection;

    private void Awake() => rb = GetComponent<Rigidbody>();

    private void Update()
    {
        float h = Input.GetAxisRaw("Horizontal");
        float v = Input.GetAxisRaw("Vertical");
        moveDirection = new Vector3(h, 0f, v).normalized;
    }

    private void FixedUpdate()
    {
        if (rb == null) return;
        rb.MovePosition(rb.position + moveDirection * moveSpeed * Time.fixedDeltaTime);
    }
}
```

Primer error real: Input

```
InvalidOperationException:  
You are trying to read Input using the UnityEngine.Input class,  
but active Input handling is Input System package.
```

Solucion para la practica: Active Input Handling = Both

- Edit > Project Settings > Player > Other Settings.
- Cambiar a Both para permitir Input.GetAxisRaw.
- Reiniciar Unity si lo pide.

Coleccionables y score

- Cada Sphere amarilla tiene Collider con Is Trigger.
- El player necesita tag Player.
- Collectible avisa a GameManager y se desactiva.
- GameManager cuenta puntos y detecta victoria.

Riesgo Unity

Si no hay Rigidbody o el tag esta mal,
OnTriggerEnter no se ejecuta como esperamos.

Collectible.cs

```
using UnityEngine;

public class Collectible : MonoBehaviour
{
    [SerializeField] private int points = 1;
    private bool collected;

    private void OnTriggerEnter(Collider other)
    {
        if (collected) return;
        if (!other.CompareTag("Player")) return;

        collected = true;

        GameManager gm = FindObjectOfType<GameManager>();
        if (gm != null) gm.AddScore(points);

        gameObject.SetActive(false);
    }
}
```


GameManager v1

Responsabilidades:

- Guardar score.
- Contar coleccionables totales al empezar.
- Actualizar ScoreText.
- Mostrar YOU WIN al recoger todo.

Restricciones:

- no eventos todavia
- no ScriptableObjects
- null-checks simples para UI
- TextMeshProUGUI en Inspector

```
public void AddScore(int points)
{
    if (gameEnded) return;

    score += points;
    collectedCount++;

    UpdateScoreUI();

    if (collectedCount ==
totalCollectibles)
        WinGame();
}
```

Tiempo, estados y reinicio

Tiempo

startTime = 60
Resta Time.deltaTime.

Estados

gameEnded bloquea score
y countdown.

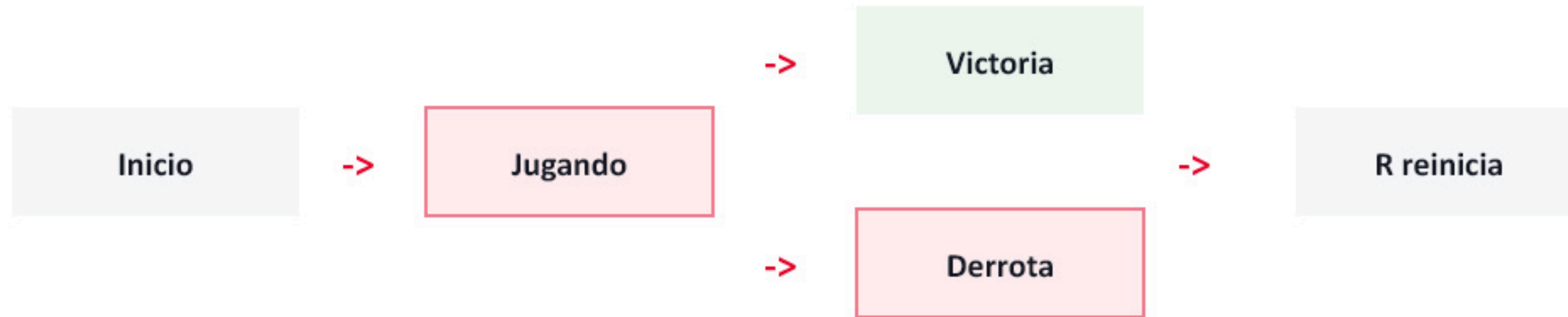
Reinicio

Tecla R + SceneManager.LoadScene.

Prompt:

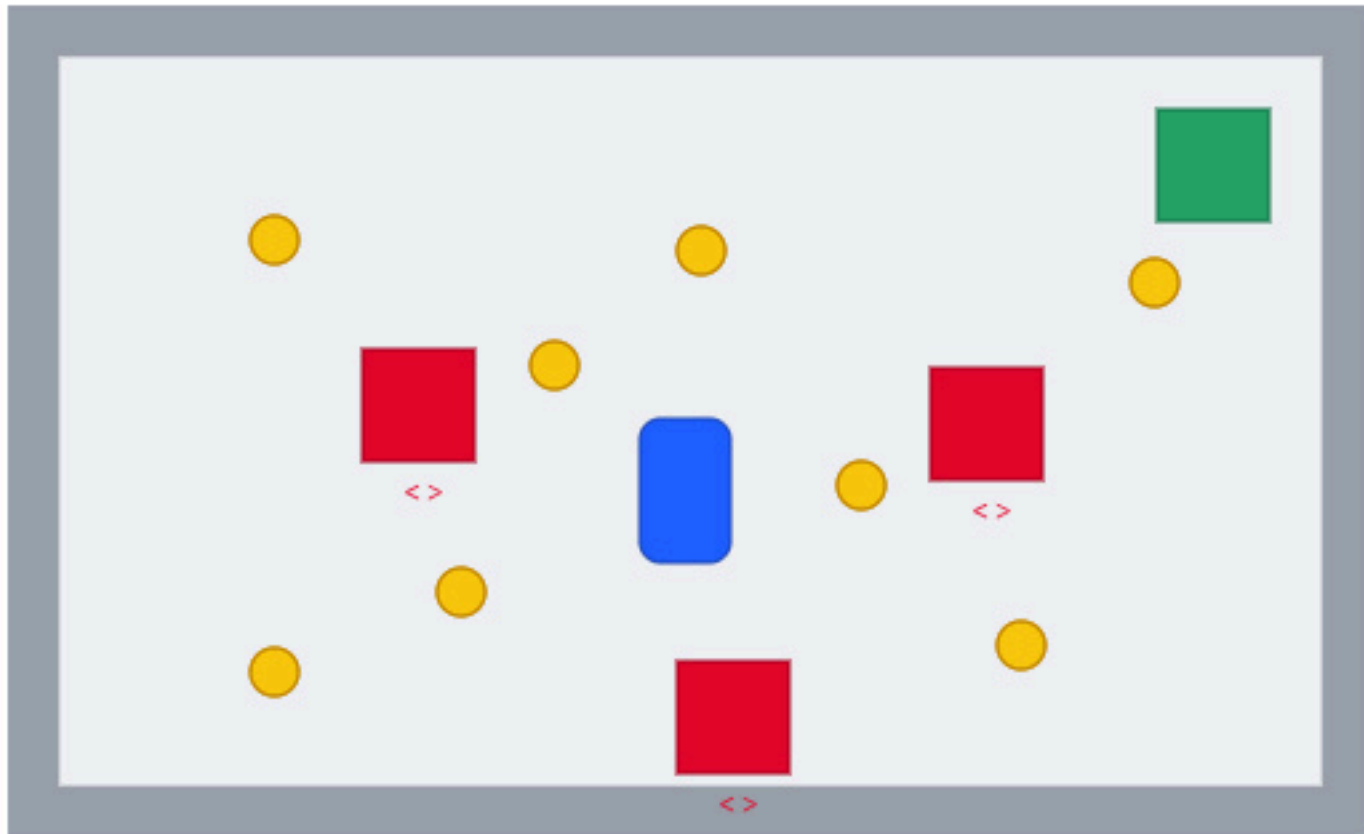
Tengo este GameManager actual: [pegar codigo]
Quiero temporizador de 60s, derrota al llegar a 0,
victoria si recoge todo antes, y reinicio con R.
Modifica solo GameManager. No uses Time.timeScale.

Flujo de estado



- El estado evita sumar puntos despues del final.
- El temporizador se detiene logicamente al ganar o perder.
- No usamos Time.timeScale para mantenerlo simple.

Obstaculos con cubos rojos



- Cubos rojos con Collider normal.
- Se mueven entre dos puntos con MoveTowards.
- Al colisionar con Player llaman a LoseGame().
- La dificultad se ajusta con velocidad, distancia y posicion.

MovingObstacle.cs

```
public class MovingObstacle : MonoBehaviour
{
    [SerializeField] private float moveDistance = 4f;
    [SerializeField] private float moveSpeed = 3f;
    [SerializeField] private Vector3 moveDirection = Vector3.right;

    private Vector3 targetA, targetB, currentTarget;
    private GameManager gameManager;

    private void Start()
    {
        Vector3 dir = moveDirection.normalized;
        targetA = transform.position - dir * moveDistance;
        targetB = transform.position + dir * moveDistance;
        currentTarget = targetB;
        gameManager = FindObjectOfType<GameManager>();
    }

    private void Update()
    {
        transform.position = Vector3.MoveTowards(transform.position, currentTarget, moveSpeed * Time.deltaTime);
        if (Vector3.Distance(transform.position, currentTarget) <= 0.05f)
            currentTarget = currentTarget == targetA ? targetB : targetA;
    }
}
```


Camara y pulido

- La opcion minima: camara fija inclinada.
- La opcion practica: CameraFollow con offset y LateUpdate.
- No usamos Cinemachine para reducir dependencias.
- Pulido: paredes, colores, posiciones, velocidades y tiempo.

```
Prompt:  
Necesito CameraFollow simple.  
Target = Capsule Player.  
Mantener offset configurable.  
Usar LateUpdate.  
No usar Cinemachine.  
Codigo completo + Inspector.
```


Debugging con ChatGPT

Tengo este error en Unity:

[pegar error completo]

Este es el script:

[pegar script]

Contexto de la escena:

- GameObject donde esta el script:
- componentes del Player:
- tag del Player:
- colliders:
- Rigidbody:
- referencias asignadas en Inspector:

Dime:

1. causa mas probable
2. como comprobarlo en Unity
3. cambio minimo
4. que no deberia tocar

Bugs tipicos de Unity

NullReferenceException

Falta asignar texto UI en Inspector.

OnCollisionEnter no salta

Collider en trigger o falta Rigidbody.

LoadScene falla

Falta using, escena no guardada o Build Settings.

OnTriggerEnter no salta

Is Trigger, tag Player o Rigidbody incorrectos.

La camara tiembla

Seguir en Update en vez de LateUpdate.

Input no funciona

Active Input Handling no esta en Both.

Alucinaciones frecuentes en Unity

- Mezclar fisica 2D y 3D: Rigidbody2D, Collider2D, OnTriggerEnter2D.
- Inventar metodos o APIs que no existen en la version del alumno.
- Usar nuevo Input System aunque la practica pida Input clasico.
- Crear managers innecesarios para una mecanica simple.
- Olvidar referencias de Inspector o cambiar nombres publicos usados por UI.

Actividad final por grupos

- Cada grupo termina una escena jugable.
- Debe explicar un prompt que le ayudo y un prompt que fallo.
- Debe mostrar una prueba en Play Mode.
- Debe identificar un riesgo de Inspector o configuracion.

Pregunta clave

Que parte fue codigo y que parte era configuracion de Unity?

Checklist de entrega

- Player se mueve y no gira raro.
- Score aumenta al recoger Spheres.
- El tiempo baja y puede provocar derrota.
- Obstaculos moviles causan Game Over.
- Victoria al recoger todo.
- Tecla R reinicia la escena.
- Textos UI asignados.
- Tag Player correcto.
- Colliders y Rigidbody revisados.
- Escena guardada.
- Build Settings revisado si hace falta.
- No hay errores en consola.

Scripts finales

PlayerMovement.cs

Input + Rigidbody.MovePosition.

Collectible.cs

Trigger + score + desactivar.

GameManager.cs

Score, tiempo, estado, reinicio.

MovingObstacle.cs

Patrulla + derrota por colision.

CameraFollow.cs

Seguimiento simple con offset.

Cinco scripts. Ningun sistema nuevo innecesario.

Cierre

La IA acelera el prototipo, no sustituye la prueba.

El flujo profesional es: describir bien el problema, pedir ayuda concreta, aplicar cambios pequenos, probar en Unity, volver con contexto y verificar antes de confiar.

Anexo: PlayerMovement completo

```
using UnityEngine;

[RequireComponent(typeof(Rigidbody))]
public class PlayerMovement : MonoBehaviour
{
    [SerializeField] private float moveSpeed = 6f;
    private Rigidbody rb;
    private Vector3 moveDirection;

    private void Awake() => rb = GetComponent<Rigidbody>();

    private void Update()
    {
        float horizontal = Input.GetAxisRaw("Horizontal");
        float vertical = Input.GetAxisRaw("Vertical");
        moveDirection = new Vector3(horizontal, 0f, vertical).normalized;
    }

    private void FixedUpdate()
    {
        if (rb == null) return;
        Vector3 nextPosition = rb.position + moveDirection * moveSpeed * Time.fixedDeltaTime;
        rb.MovePosition(nextPosition);
    }
}
```

Anexo: Collectible completo

```
using UnityEngine;

public class Collectible : MonoBehaviour
{
    [SerializeField] private int points = 1;
    private bool collected;

    private void OnTriggerEnter(Collider other)
    {
        if (collected) return;
        if (!other.CompareTag("Player")) return;

        collected = true;

        GameManager gameManager = FindObjectOfType<GameManager>();
        if (gameManager != null)
            gameManager.AddScore(points);

        gameObject.SetActive(false);
    }
}
```

Anexo: GameManager 1/3

```
using TMPro;
using UnityEngine;
using UnityEngine.SceneManagement;

public class GameManager : MonoBehaviour
{
    [SerializeField] private TextMeshProUGUI scoreText;
    [SerializeField] private TextMeshProUGUI timeText;
    [SerializeField] private TextMeshProUGUI messageText;
    [SerializeField] private float startTime = 60f;

    private int score;
    private int totalCollectibles;
    private int collectedCount;
    private float timeRemaining;
    private bool gameEnded;

    private void Start()
    {
        totalCollectibles = FindObjectsOfType<Collectible>().Length;
        timeRemaining = startTime;
        UpdateScoreUI();
        UpdateTimeUI();
        if (messageText != null) messageText.text = "";
    }
}
```


Anexo: GameManager 2/3

```
private void Update()
{
    if (Input.GetKeyDown(KeyCode.R)) { RestartGame(); return; }
    if (gameEnded) return;

    timeRemaining -= Time.deltaTime;
    if (timeRemaining <= 0f)
    {
        timeRemaining = 0f;
        UpdateTimeUI();
        LoseGame();
        return;
    }
    UpdateTimeUI();
}

public void AddScore(int points)
{
    if (gameEnded) return;
    score += points;
    collectedCount++;
    UpdateScoreUI();
    if (collectedCount >= totalCollectibles) WinGame();
}
```

Anexo: GameManager 3/3

```
public void LoseGame()
{
    if (gameEnded) return;
    gameEnded = true;
    if (messageText != null) messageText.text = "GAME OVER - Press R";
}

private void WinGame()
{
    gameEnded = true;
    if (messageText != null) messageText.text = "YOU WIN! - Press R";
}

private void RestartGame()
{
    Scene currentScene = SceneManager.GetActiveScene();
    SceneManager.LoadScene(currentScene.name);
}

private void UpdateScoreUI() { if (scoreText != null) scoreText.text = "Score: " + score; }
private void UpdateTimeUI() { if (timeText != null) timeText.text = "Time: " + Mathf.CeilToInt(timeRemaining); }
}
```

Anexo: MovingObstacle + CameraFollow

```
// MovingObstacle: derrota al tocar Player
private void OnCollisionEnter(Collision collision)
{
    if (!collision.collider.CompareTag("Player")) return;
    if (gameManager != null) gameManager.LoseGame();
}

// CameraFollow: seguimiento simple
private void LateUpdate()
{
    if (target == null) return;

    Vector3 desired = target.position + offset;
    transform.position = Vector3.Lerp(transform.position, desired, smoothSpeed * Time.deltaTime);
    transform.LookAt(target);
}
```

En aula conviene mostrar los scripts completos en Unity y usar estas diapositivas como referencia proyectable.